전북대학교
JEONBUK NATIONAL UNIVERSITY

유닉스 시스템 프로그래밍
(Unix System Programming)

2023 - 1 학기

16. Thread

Jongwook Jeong
정 종 욱

Division of Computer Science and Engineering

Thread

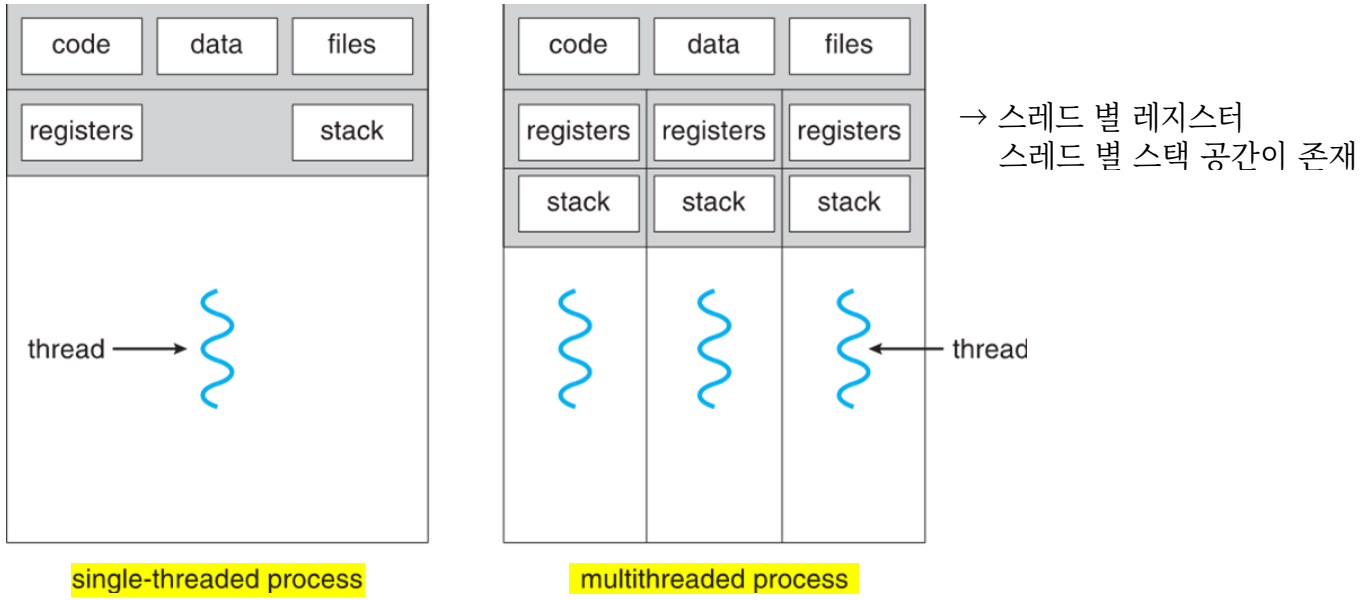
Thread

- 두 가지 종류의 실행 주체(execution unit or entity)
  - Process: 독립적인 메모리 공간을 가진 수행 주체
    - 각각 독립적인 메모리 공간을 생성해주고, 관리해주어야 하기 때문에
    - 생성/제거에 시간이 오래 걸리고, 시스템 자원을 많이 사용함
  - Thread: 프로세스의 단점을 보완하기 위해 새로이 정의된 실행 주체

- Thread: "The execution unit in a process"
  - Thread = CPU Register set + Independent Stack
    - 하나의 수행 흐름이 다른 수행 흐름과 분리되기 위해 필수적인 요소들만 모아 Thread로 정의
  - 하나의 프로세스 내에 여러 쓰레드가 함께 동작할 수 있음 → 멀티 스레드
    - 쓰레드들은 프로세스의 자원을 공유해서 사용: code, data, heap 영역, File descriptor 등
  - 장점
    - 프로세스보다 가볍다 (a.k.a. LWP: light-weight process)
      - 여러 쓰레드가 동시에 동작할 때, 여러 프로세스의 경우보다 효율적임 (성능 높음)
    - IPC 가 불필요함: 하나의 프로세스 내에서 메모리 공간을 공유하기 때문
  - 단점: 데이터 공유에 따른 동기화 문제, 오류 전파 (error propagation) 등

Thread

- Single-threaded Process vs. Multi-threaded Process
  - STP: 전통적인 프로세스의 구조와 동일함
    - 내부에서 명시적으로 쓰레드 관련 서비스를 사용하지 않더라도,
    - 코드가 수행되는 하나의 흐름을 쓰레드라는 개념으로 구체화하였음
    - 프로세스 생성이 완료되면, 기본 쓰레드가 코드를 순차적으로 수행하기 시작함
  - MTP: 쓰레드 개념이 등장하면서 가능해진 구조
    - 여러 쓰레드가 하나의 프로세스 내에서 프로세스의 자원을 공유하며 동시에 수행됨
- "프로세스" 는 더 이상 실행 주체 라기보다, 실행 주체인 쓰레드를 담는 그릇과 같은 역할



Pthread 서비스

- Pthread (POSIX Thread)
  - POSIX 는 운영체제 API 표준으로, 여러 OS에서 지원하고 있음
  - pthread 라이브러리: POSIX 에서 쓰레드를 관리하기 위한 여러가지 API를 구현한 것
    - 쓰레드 관련 시스템 콜들을 대신 호출하고, 관련 자료 구조를 관리함
  - 가장 일반적으로 널리 이용되는 쓰레드 라이브러리
  - 생성, 종료, 완료 대기, 동기화 관리 (lock) 등의 기능을 제공
  - gcc 컴파일 시, "-l pthread" 옵션을 사용해 명시적으로 라이브러리 지정을 해주어야 함

	프로세스	쓰레드
생성	fork()	pthread_create()
완료 대기	wait()	pthread_join()
수행 코드 변경	exec()	-

쓰레드 생성: pthread_create()

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);

Compile and link with -pthread.
```

- thread: 생성된 쓰레드를 가리키는 구조체. 쓰레드를 가리키기 위한 ID로 사용
  - TID: Thread ID. 숫자가 아니라 구조체로 표현됨
  - pthread_self()를 통해 확인 가능. pthread_equal()을 통해 비교 가능
- attr: struct pthread_attr_t 로 정의된 쓰레드의 속성들. NULL 인 경우, 기본 속성
- start_routine: 쓰레드로 수행할 코드의 함수 포인터
  - void * start_thread (void *arg);
- arg: start_routine 에 전달할 인자
- Return value
  - On success: 0 , On error: error number (not zero)

쓰레드의 종료

- 쓰레드를 종료하는 네 가지 방법
  - 쓰레드 생성 시 지정된 start_routine 에서 return 하는 경우
  - pthread_exit() : 자기 자신을 종료
    - 상태 종료값을 retval 로 전달. retval 은 동적으로 할당하여 사용
  - pthread_cancel() : 인자로 지정된 쓰레드를 종료
  - 전체 프로세스가 종료되는 경우
    - main() 함수의 return
    - (어느 thread 에서든) exit() 호출
    - exec() 시리즈로 새로운 코드가 로드된 경우

```
void pthread_exit(void *retval);
```

```
int pthread_cancel(pthread_t thread);
```

쓰레드 종료 대기: pthread_join()

```
#include <pthread.h>

int pthread_join(pthread_t thread, void **retval);
```

- thread 의 종료를 대기하고, retval 에 종료 상태값을 전달함
  - 프로세스에서의 wait() 와 유사한 역할
- Return value
  - On success: 0 , On error: error number (not zero)

- pthread_detach
  - 만약 쓰레드를 join 하지 않기로 결정했다면,
  - detach 를 수행하여 쓰레드 종료 시, 즉각 쓰레드를 destroy 하도록 함
  - 예) 프로세스의 경우, wait()를 수행하지 않으면 좀비 프로세스가 됨
    - 쓰레드는 detach 를 시킨 경우, 다른 쓰레드가 join 을 수행하지 않아도 무방함

```
int pthread_detach(pthread_t thread);
```

[예제] 1~5 숫자 세기 (multi) 쓰레드

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define MAX_THREADS (10)


pthread_t tid[MAX_THREADS];

void* counting (void *arg){
        //쓰레드가 실행되는 함수

        int i, indent = *((int *)arg), *ret;
        char buf[80];
        //indent는 전달받은 값 (1번 쓰레드 = 1, 2번쓰레드 = 2...)


        srand((unsigned int)indent);
        sleep(rand()%3);

        for(i=0; i<indent; i++)buf[i] = '\t';
        //전달받은 값 만큼 buf에 들여쓰기로 공백 채워줌

        for(i=0;i<5;i++){
                printf("%s%d...\n", buf, i);
                sleep(1);
        } //초시계 (1초마다 카운트) 시작

        printf("%sFINISHED!\n", buf);

        ret = (int *)malloc(sizeof(int));
        *ret = indent;
        pthread_exit(ret);
}
```

```c
int main(int argc, char *argv[]){
        int i, threads, *status, tcounts = 0, args[MAX_THREADS];
        //i는 반복문을 위한 인자
        //threads는 쓰레드의 갯수 (입력값)
        //tcounts는 생성된 쓰레드의 갯수


        if(argc != 2){
                printf("Usage: ./thread <number of thread>\n");
                return 1;
        }

        threads = atoi(argv[1]);

        if(threads < 1 || threads > MAX_THREADS){
                printf("Max. number of thread is %d\n",MAX_THREADS);
                return 2;
        }

        printf("Your input: %d\n", threads);

        for(i=0; i<threads; i++, tcounts++){
                args[i] = i;
                if (pthread_create(&tid[i], NULL, counting, &args[i]) != 0){
                        //tid는 앞서 정의한 쓰레드 구조체의 배열, NULL은 쓰레드의 속성 옵션(기본이 NULL)
                        //쓰레드가 실행하는 코드의 함수 포인터는 counting, args[i]의 값이 각 스레드에 인자로 전달
                        perror("Failed to create thread");
                        goto exit;
                }
        }

exit:
        for(i=0;i<tcounts;i++){
                pthread_join(tid[i], (void **) &status);
                printf("Thread no.%d ends: %d\n", i, *status);
        }

        return 0;
}
```

[예제] 수행 결과

```
root@jjw:/home/jjw# ./thread 3
Your input: 3
                0...
        0...
0...
                1...
        1...
                2...
1...
        2...
                3...
2...
        3...
                4...
3...
        4...
                FINISHED!
4...
        FINISHED!
FINISHED!
Thread no.0 ends: 0
Thread no.1 ends: 1
Thread no.2 ends: 2
```

```
root@jjw:/home/jjw# ./thread 7
Your input: 7
                                        0...
                        0...
                0...
                                0...
                                        1...
                        0...
                1...
        0...
0...
                                1...
                                        2...
                        1...
                2...
        1...
1...
                                2...
                                        3...
                        2...
                3...
        2...
2...
                                3...
                                        4...
                        3...
                4...
        3...
3...
                                4...
                                        FINISHED!
                        4...
                FINISHED!
        FINISHED!
4...
                                FINISHED!
                        FINISHED!
FINISHED!
Thread no.0 ends: 0
Thread no.1 ends: 1
Thread no.2 ends: 2
Thread no.3 ends: 3
Thread no.4 ends: 4
Thread no.5 ends: 5
Thread no.6 ends: 6
```